

Chapter 3: Hello World

It is a tradition in computer science that every time you learn a new programming language your first application is “Hello World.” We of course will adhere to that tradition. It is simply way too important.

The reason everybody writes the “Hello World” app is because it teaches you how to write, compile, run and debug an application in an otherwise totally new environment. In our case that new environment will be Microsoft’s Visual Studio.

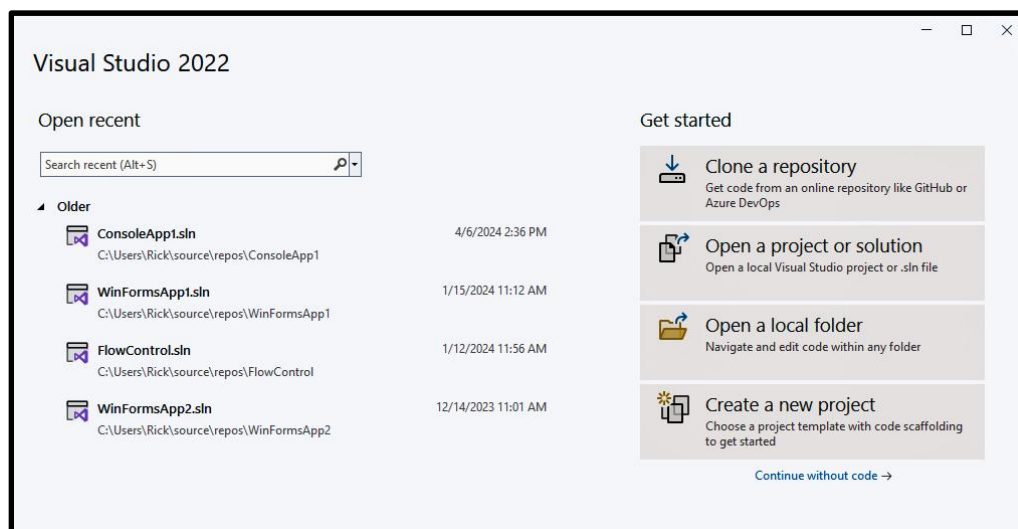
Okay, let’s learn how to write, compile, execute and debug our very first C# application.

This is the home screen for Visual Studio 2022. (Figure 3.1) Remember our launch icon for that guy looks like the one shown here.



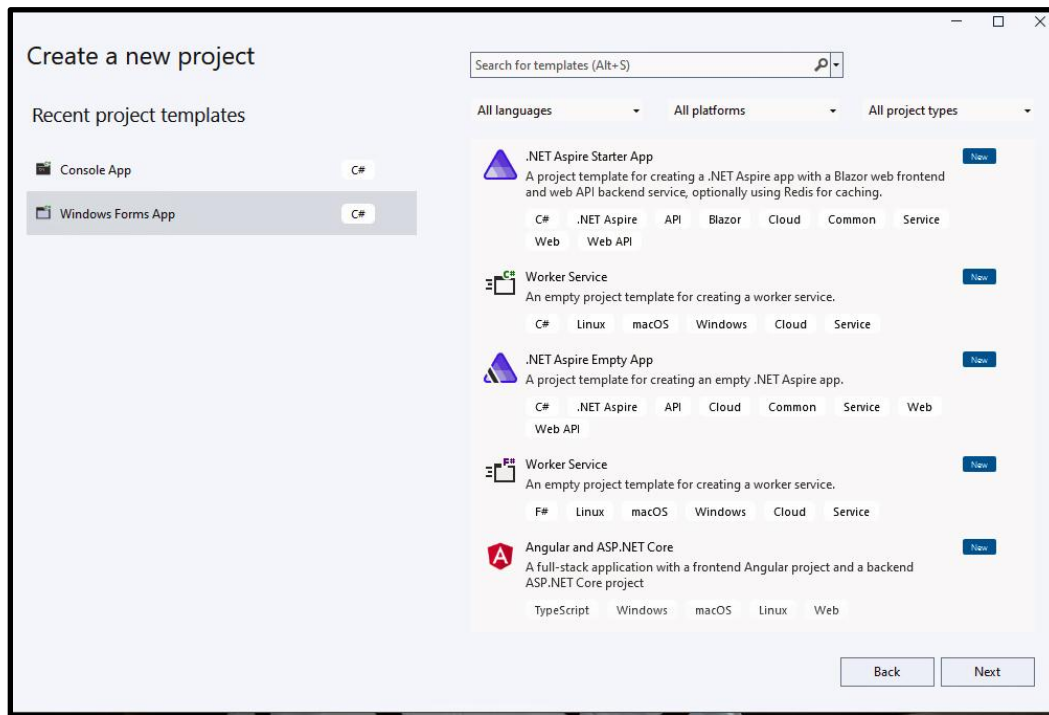
Your screen will not have the older projects shown on the left because this is your first application. As you write more applications you will get a list of older projects as well. To jump right into one of those older projects simply double click on its name.

Figure 3.1: Visual Studio Home Screen



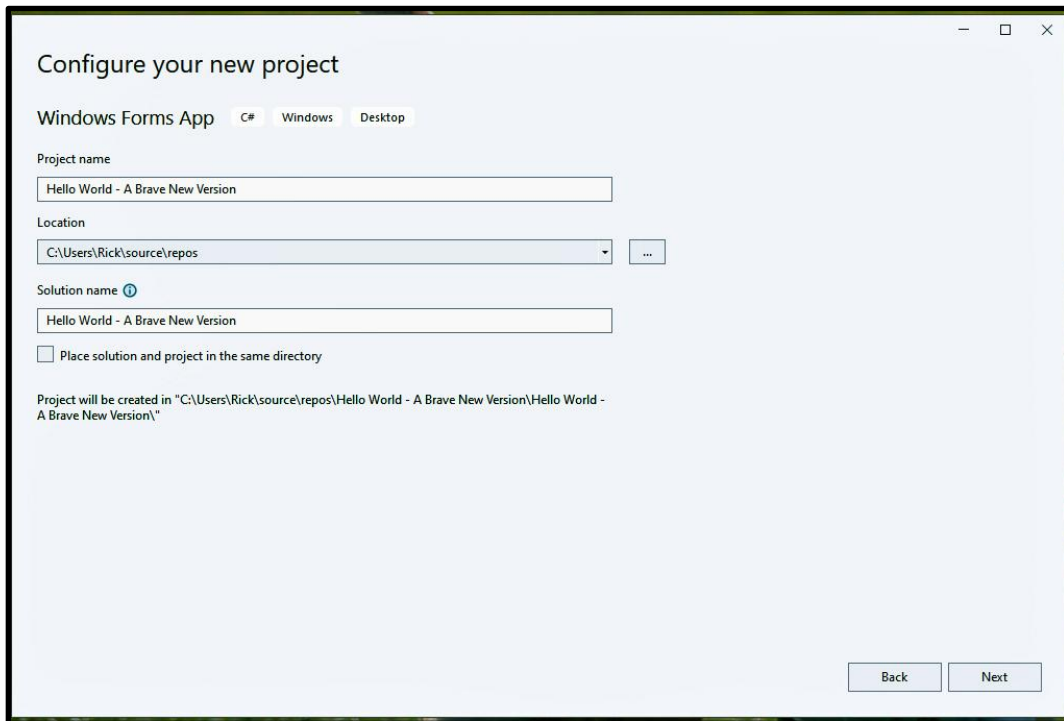
We of course are going to create a new project so please just go ahead and click on that option. This image depicts our next screen. (Figure 3.2) We are going to create a C# Windows Forms App so please click on that option next. Note, you may need to scroll down quite a way to get to that option.

Figure 3.2: Project Type Options Screen



The image shown here is our last screen before we create our project. (Figure 3.3) Please go ahead and add the Project name as “Hello World – A Brave New Version.” By default, Visual Studio will copy that name to the Solution Name field, go ahead and accept that default and click the Next button. Just as an FYI, a solution is a collection of one or more projects, so you can think of a project as a subset of a solution. In our case, this solution will have only one project, the one we named “Hello World – A Brave New Version.”

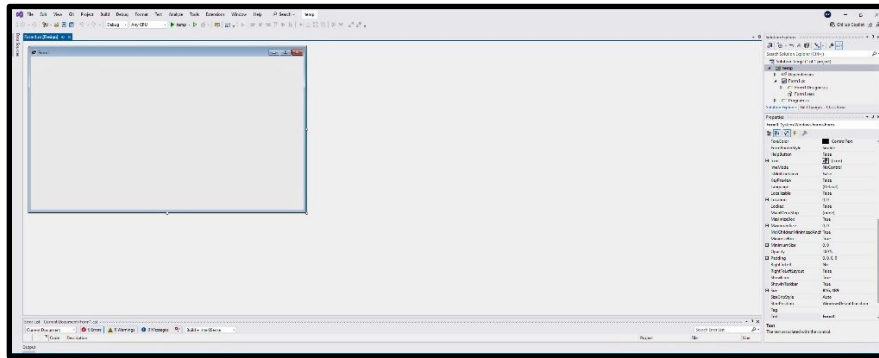
Figure 3.3: Project/Solution Name



The screenshot shows a 'Configure your new project' dialog box. At the top, it says 'Configure your new project' with standard window controls. Below that, it identifies the project type as 'Windows Forms App' with tabs for 'C#', 'Windows', and 'Desktop'. The 'Project name' field contains 'Hello World - A Brave New Version'. The 'Location' field shows a file explorer icon and the path 'C:\Users\Rick\source\repos'. The 'Solution name' field also contains 'Hello World - A Brave New Version'. There is an unchecked checkbox labeled 'Place solution and project in the same directory'. At the bottom, it states 'Project will be created in "C:\Users\Rick\source\repos\Hello World - A Brave New Version\Hello World - A Brave New Version\'". Navigation buttons 'Back' and 'Next' are at the bottom right.

At this point Visual Studio may ask you which version of the .NET Framework to use with your project. Just go ahead and accept the default.

The very scary screen shown here should appear next. (Figure 3.4) Before you completely freak out, let's look at some of the various components of Visual Studio's Integrated Development Environment or IDE.



We start with the large white area in the top left-hand corner. This is the graphic design area and Visual Studio has already added a Form to that area allowing for the start of a display window. Please note that the Form is labeled “Form1” in the upper left-hand corner. That is because the Form’s Text property is set to “Form1.” I know that because if I look in the Properties pane in the screen’s bottom right-hand corner, I see the word “Form1” next to the Text property of the Form. Properties are graphic characteristics of an object. If your environment does not currently include the Solution Explorer and Properties panes as shown here, go ahead and enable them via the View menu option. In addition, if Copilot is taking up a bunch of your screen geography go ahead and disable that guy as we are sure not ready for any AI assistance at this point.

Our only object is our Form, and the property we want to consider is its Text value. Let's go ahead and change the value of that guy to Hello World. If the properties are not listed in alphabetical order, you can change that by clicking the little icon with the A-Z label at the top of the pane. That is my preferred way of viewing object properties. So, what happened when you changed the Text property of our Form to Hello World? Sure, the text value in the upper left-hand corner of our Form changed.

Anybody want to guess what happens when we change the Size property of our Form? Go ahead and temporarily make it 200,150 and see what happens. Okay, as you can see the Size property determines the pixel width by height of our Form. Please go ahead and permanently set that guy to 500,300 so we have some geography to work with going forward.

Next, I want you to go over to the far left-hand side of the screen and look for the word toolbox. If it is not there, again go up to the View menu at the top of the screen and select the toolbox option. When open, select the All Windows Forms option and then read down the list of objects until you see the word Label. Double click that guy to add a Label object to your Form. Next, go ahead and dock the toolbar to the left-hand side of your work environment.

Next click back on the Form to see our newly added Label object. Cool, there it is in the upper left-hand corner. Anybody want to guess how we move that Label to the middle of the screen? Remember that an object's properties determine how it appears on the screen. Let's look for a property that might change the Label's location. All right, I bet the Location property determines the x/y pixel location of our Label's upper left-hand corner relative to the upper left-hand corner of the Form. Let's change that value to 100,100 and see what happens. Okay, that looks pretty good.

Next let's change the Text of our label to read Hello World. Not surprisingly, I am going to do this by updating the Text property of my Label. To finish off this part of our Form, let's go ahead and change the font color (which is actually the ForeColor property) to red. Okay, that looks pretty good so let's stop here for a couple of minutes.

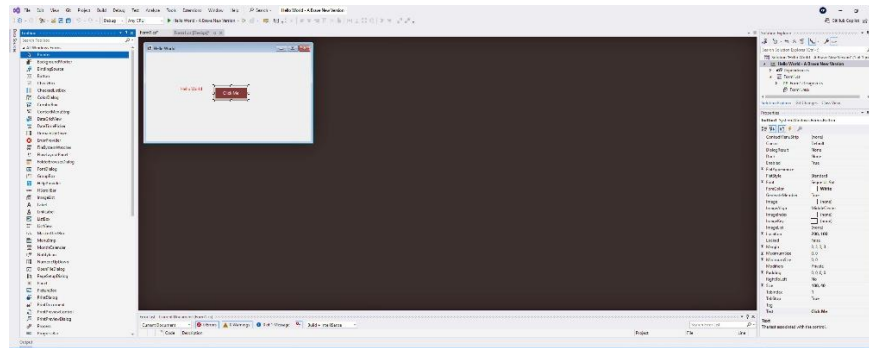
At this point we actually have a Form which is ready to compile and test. A compiled application is one which has been transformed into machine executable commands. Compiled programming languages like C# and Java vastly outperform interpreted languages like Python and Basic and as such are much more appropriate for large-scale solutions.

To compile and then execute our application go ahead and click that green arrow up at the top of the screen. Visual Studio will automatically compile and then begin to run your application. By default, the debugger will also start. We will look at the debugger in detail later in this chapter. For now, simply click the green arrow.

Next, I want to teach you how to save and reopen a project in Visual Studio. Super easy. To save a project just hit the double floppy disk icon in the taskbar. By the way, the single floppy disk saves just your current Form if you are working with several.

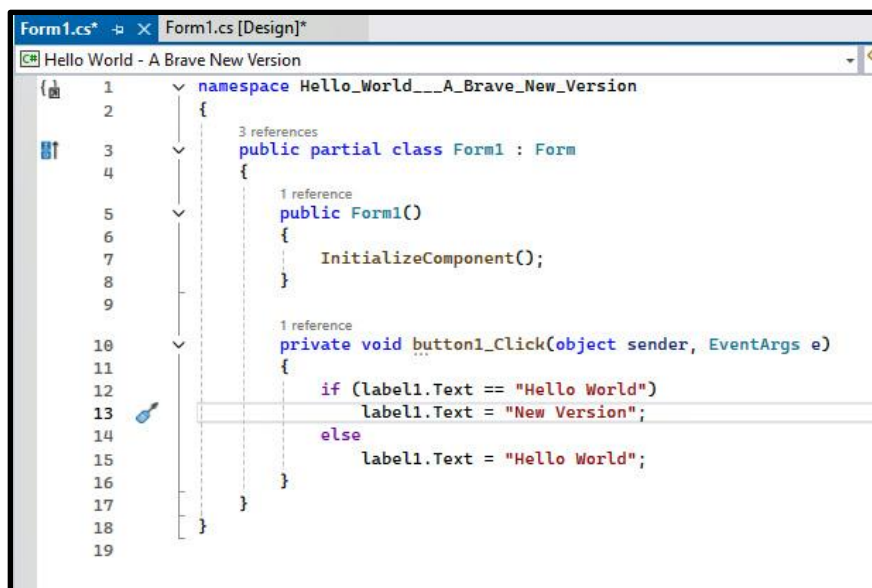
We are now ready to make this Form actually do something. Let's begin by adding a Button to our Form. Set the Size property of your Button to 100,40 and it's Text property to Click Me. Let's go ahead and change its Location property to 200, 100, it's background color (where the actual property name is BackColor) to Brown and its font color (remember that property name is actually ForeColor) to White. Okay, that all looks good. Let's run a quick test before we write our code. (Figure 3.5)

Figure 3.5: Hello World Form



Okay, we are now going to make some magic happen. I want you to first double click that Button we just created. A new screen opens and your cursor should be located inside an event handler named `button1_Click`. Let's not worry about all the other stuff on this screen for right now, let's just work inside this event handler. (Figure 3.6)

Figure 3.6: `button1_Click` Event Handler



First, what is an event handler. Events are actions initiated by the user. In this case this event handler will execute when the user clicks on the Button we added to our Form. Objects have tons of events to which they may respond. The default event for a Button is a click hence the default name given to this handler.

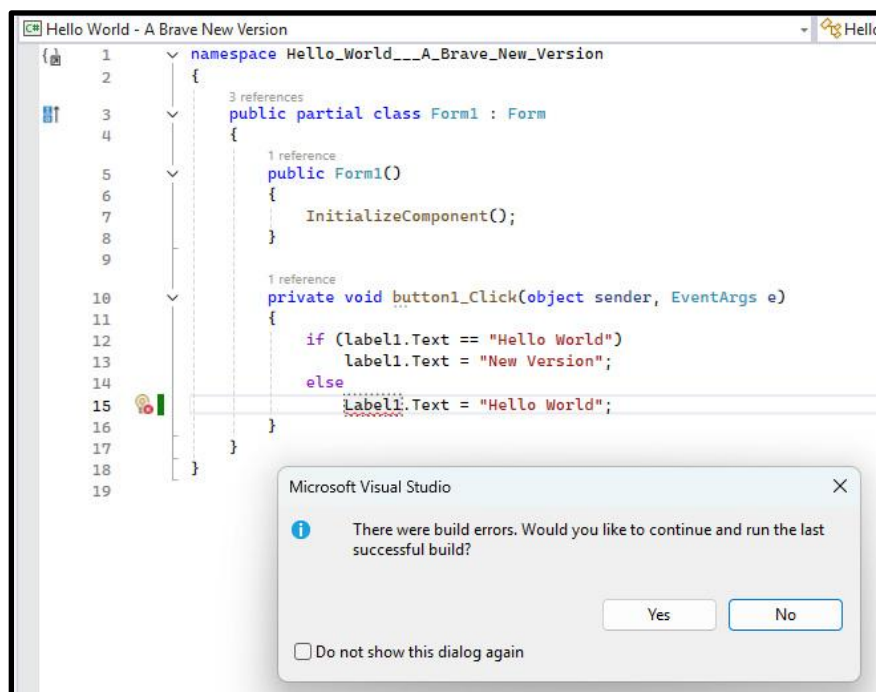
Now I want you to type this next couple of lines of code very carefully to ensure no errors are introduced.

```
if (label1.Text == "Hello World")  
    label1.Text = "New Version";  
else  
    label1.Text = "Hello World";
```

A couple of things to know about this code. First, it is your very first C# code so congratulations once you get it working. Second, the if else syntax is known as flow control. Flow control governs which lines of syntax get executed and how often. In our case here the syntax says if the Text property of label1 reads Hello World change it to New Version. If it does not read Hello World, a.k.a. the else condition, then set the Label's Text property value back to Hello World.

So how many of you got it to work on the first iteration? My guess is not too many. C# is very syntax bound, and it is very easy to make a typing error when you are first starting out. For example, did you notice that the equality test required two equal signs? How many of you started your label1 name with a capital letter L? C# is case sensitive so Label1 is not equivalent to label1 and the compiler will fuss at you if you make a mistake. What is the good news, syntax errors are typically very easy to catch because the compiler is going to show you right where your issue is located. In my example here the compiler has underlined my incorrectly formed Label name in red to point out my mistake. I don't need to trash my work, just fix the mistake. (Figure 3.7)

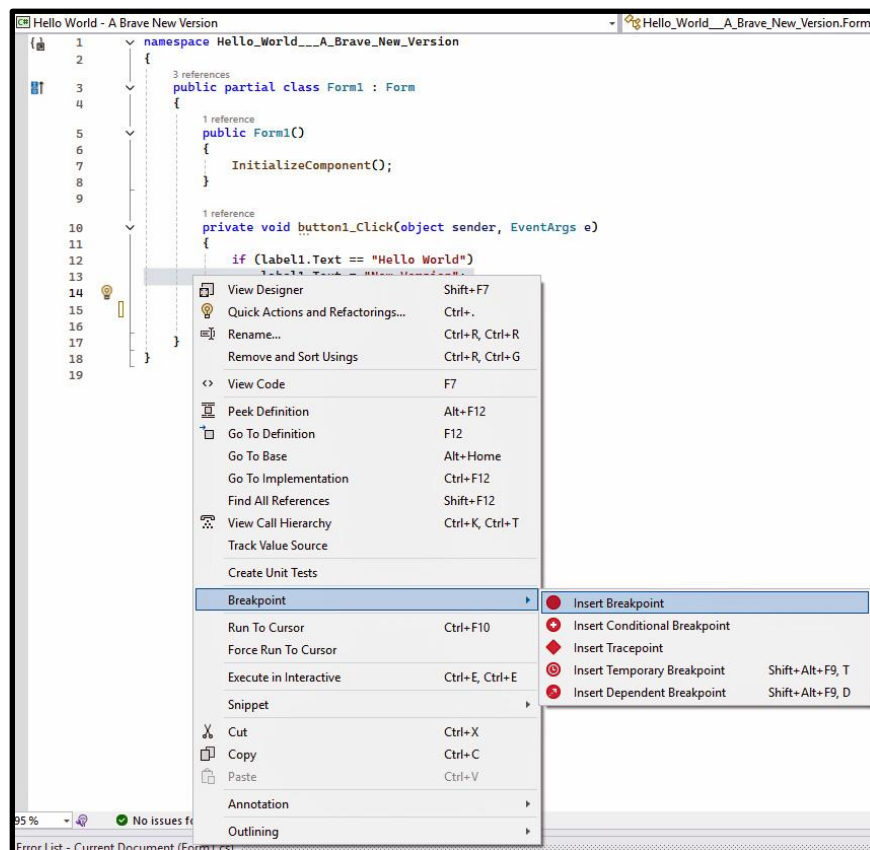
Figure 3.7: Compile Errors



By the way, did you notice that you now have two tabs up above my work area? One is labeled Form1.cs [design]. That is where the Form and its associated objects live, i.e. it is the design environment. The one labeled just Form1.cs is where the C# code lives.

But what if it didn't work? What can I do to see what is going on behind the scenes of my application. You really have two options. `MessageBox.Show` stops the current execution of your program and displays a message to the user. During development that message is typically some information which you don't think is behaving as you expect. The second option is to add a breakpoint. To add a breakpoint to a line of code, simply right click on the line and select the appropriate breakpoint option. (Figure 3.8) When a breakpoint is present in your code the execution stops at that point, and you can examine all the variables and objects for potential issues.

Figure 3.8: Breakpoints



All right, that wraps up our little excursion into the Hello World application. We learned how to create a project in Visual Studio, add objects to a form, compile and execute an application and even write a little code behind our Form. Debugging is more art than science, but `MessageBox.Show` and breakpoints help us look for and identify issues.